

[main, import]german

Technischer Bericht

# Subgraph-basierte Optimierung von KI-Rechenzentrumsinfrastrukturen

Anwendung des Subgraph Algorithmus auf die  
Ressourcenplanung, Workload-Platzierung  
und CPU:GPU-Verhältnisanalyse  
am Beispiel des T-Systems Rechenzentrums Bielefeld

Stephan Epp

Universität Bielefeld

<https://github.com/hjstephan86/science>

11. Mai 2026

## Abstract

Moderne KI-Rechenzentren stehen vor der algorithmischen Herausforderung, heterogene Workload-Abhängigkeitsgraphen optimal auf physische Infrastrukturgraphen abzubilden. Dieser Bericht zeigt, dass der Subgraph Algorithmus [1] — ein  $O(n^3)$ -Algorithmus zum zyklischen Graphvergleich mittels Signatur-Arrays und LCS-Matching — direkt als Kernkomponente eines ressourcenoptimalen Workload-Schedulers eingesetzt werden kann.

Wir modellieren das T-Systems-Rechenzentrum Bielefeld (Detmolder Straße 380) als Topologiegraph  $G_{RZ}$  und jede KI-Aufgabe als Workload-Abhängigkeitsgraphen  $W_i$ . Das *RZ-Placement-Problem* wird formal als zyklisches Subgraph-Isomorphie-Problem definiert.

**Erweiterung (Mai 2026):** Diese Fassung enthält eine vollständige Integration der CPU:GPU-Verhältnisanalyse [2] – von der formalen Modellierung der Hardware-Topologiegraphen  $G_r$  ( $r \in \{1 : 1, 1 : 2, 1 : 4, 1 : 8\}$ ) über den Beweis der Einbettungshierarchie  $G_{1:1} \hookrightarrow G_{1:2} \hookrightarrow G_{1:4} \hookrightarrow G_{1:8}$  bis hin zur quantitativen Effizienzbewertung, Prognosemodellen und ökonomischen Konsequenzen. Wir beweisen, dass (a) der Subgraph Algorithmus das RZ-Placement-Problem korrekt löst, (b) er unter SETH optimal ist mit Laufzeit  $\Theta(n^3)$ , (c) eine GPU-parallele Implementierung Echtzeit-Placement für  $n \leq 512$  ermöglicht, und (d) die optimale CPU:GPU-Konfiguration workload-abhängig formal bestimmbar ist.

**Schlüsselwörter:** Subgraph-Isomorphismus, Rechenzentrum-Scheduling, KI-Workload-Placement, CPU:GPU-Verhältnis, Signatur-Arrays, LCS, SETH-Optimalität, GPU-Parallelisierung, Fat-Tree-Topologie, Hardware-Topologiegraphen, Einbettungshierarchie

# Contents

|           |                                                                       |           |
|-----------|-----------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Einführung</b>                                                     | <b>4</b>  |
| 1.1       | Motivation . . . . .                                                  | 4         |
| 1.2       | Problemstellung . . . . .                                             | 4         |
| 1.3       | Beitrag dieser Arbeit . . . . .                                       | 4         |
| <b>2</b>  | <b>Grundlagen</b>                                                     | <b>5</b>  |
| 2.1       | Graphtheoretische Definitionen . . . . .                              | 5         |
| 2.2       | Der Subgraph Algorithmus . . . . .                                    | 6         |
| 2.3       | Rechenzentrum-Topologien . . . . .                                    | 6         |
| <b>3</b>  | <b>Formale Modellierung</b>                                           | <b>7</b>  |
| 3.1       | KI-Workload-Graphen . . . . .                                         | 7         |
| 3.2       | Das RZ-Placement-Problem . . . . .                                    | 8         |
| <b>4</b>  | <b>Der Subgraph Algorithmus im RZ-Kontext</b>                         | <b>9</b>  |
| 4.1       | Algorithmus-Beschreibung . . . . .                                    | 9         |
| 4.2       | Korrektheit . . . . .                                                 | 9         |
| <b>5</b>  | <b>Komplexitätsanalyse</b>                                            | <b>9</b>  |
| 5.1       | Laufzeit und SETH-Optimalität . . . . .                               | 9         |
| <b>6</b>  | <b>GPU-Parallelisierung</b>                                           | <b>10</b> |
| <b>7</b>  | <b>Experimentelle Projektionen</b>                                    | <b>11</b> |
| <b>8</b>  | <b>CPU:GPU-Verhältnisanalyse</b>                                      | <b>12</b> |
| 8.1       | Motivation: Das 1:1-Paradigma und seine Konsequenzen . . . . .        | 12        |
| 8.2       | Formale Modellierung der Hardware-Topologiegraphen . . . . .          | 13        |
| 8.3       | Subgraph-Beziehungen und Einbettungshierarchie . . . . .              | 15        |
| 8.3.1     | Detaillierter Nachweis: $G_{1:1} \subseteq G_{1:2}$ . . . . .         | 15        |
| 8.3.2     | Nachweis: $G_{1:2} \subseteq G_{1:4}$ . . . . .                       | 16        |
| 8.3.3     | Allgemeiner Hierarchiesatz . . . . .                                  | 16        |
| 8.3.4     | Vollständige Paarvergleich-Tabelle . . . . .                          | 16        |
| 8.4       | Komplexitätsanalyse nach Konfiguration . . . . .                      | 17        |
| 8.5       | Speicherbandbreite, Roofline-Modell und Effizienzanalyse . . . . .    | 18        |
| 8.6       | Formales Optimierungsproblem für das T-Systems RZ Bielefeld . . . . . | 20        |
| 8.7       | Integration mit dem Subgraph-Scheduling-Framework . . . . .           | 20        |
| 8.8       | Signatur-Analyse der Hardware-Topologiegraphen . . . . .              | 21        |
| 8.9       | CPU-Nachfrageprognose und ökonomische Konsequenzen . . . . .          | 22        |
| <b>9</b>  | <b>Zusammenfassung</b>                                                | <b>23</b> |
| <b>10</b> | <b>Ausblick</b>                                                       | <b>24</b> |
| 10.1      | Hardware-Architektur: CPU:GPU-Integration bis 2030 . . . . .          | 24        |
| 10.1.1    | Heterogene Compute-Fabric und 3D-Stacked Compute . . . . .            | 24        |
| 10.1.2    | In-Memory Computing und CXL . . . . .                                 | 25        |
| 10.2      | Erweiterte Algorithmenentwicklung . . . . .                           | 25        |
| 10.2.1    | Dynamische Topologien und Streaming-Variante . . . . .                | 25        |

|        |                                                                          |    |
|--------|--------------------------------------------------------------------------|----|
| 10.2.2 | Parallele Subgraph-Isomorphismus-Erkennung . . . . .                     | 25 |
| 10.2.3 | KI-Beschleuniger jenseits von CPU und GPU . . . . .                      | 25 |
| 10.3   | Integration in die T-Systems Industrial AI Cloud . . . . .               | 25 |
| 10.4   | Federated Learning, Datensouveränität und formale Verifikation . . . . . | 25 |
| 10.5   | Quantencomputing-Erweiterung . . . . .                                   | 26 |
| 10.6   | Offene Forschungsfragen . . . . .                                        | 26 |
| 10.7   | Langfristige Perspektive: Bielefeld als KI-Hub OWL . . . . .             | 26 |

# 1 Einführung

## 1.1 Motivation

Das Rechenzentrum von T-Systems an der Detmolder Straße 380 in Bielefeld-Stieghorst ist ein historisch gewachsener ICT-Knoten der Deutschen Telekom und betreibt heute eine Data-Center-Plattform im europaweiten Netz von T-Systems [3]. Mit dem industriepolitischen Beschluss der Bundesregierung, die KI-Rechenkapazitäten in Deutschland bis 2030 zu vervierfachen [5], stellt sich für den Bielefelder Standort die Frage nach einer algorithmisch fundierten Optimierungsstrategie.

Das zentrale Problem: Wenn ein Rechenzentrum zukünftig sowohl klassische Managed-Services-Workloads als auch KI-Trainings- und Inferenz-Workloads auf einer gemeinsamen physischen Infrastruktur betreiben soll, dann muss jeder eingehende Job als gerichteter Abhängigkeitsgraph  $W = (V_W, E_W)$  auf einen geeigneten Teilgraphen der physischen Topologie  $G_{RZ} = (V_{RZ}, E_{RZ})$  abgebildet werden. Dieses Problem ist in seiner allgemeinen Form NP-schwer (Subgraph-Isomorphismus [7]); für die strukturierten Fat-Tree-Topologien moderner Rechenzentren und KI-Workloads mit typischer DAG-Struktur zeigen wir jedoch, dass der Subgraph Algorithmus [1] eine  $\Theta(n^3)$ -Lösung liefert.

**Neue Perspektive (Mai 2026):** Nvidia, Meta und – seit dem 9. Mai 2026 – auch AMD propagieren aktiv das 1:1-CPU:GPU-Verhältnis als neue Norm für bestimmte KI-Workloads [2]. Gleichzeitig bleibt das 1:4-Format für Trainingsszenarien mit hohem Batch-Durchsatz weiterhin relevant, wie AMDs Helios-Rack mit  $1 \times$  Venice-CPU und  $4 \times$  MI455X belegt. Diese Entwicklung hat direkte Konsequenzen für das RZ-Bielefeld: Das CPU:GPU-Verhältnis bestimmt die Struktur der Hardware-Topologiegraphen, auf die der Subgraph Algorithmus angewendet wird – und damit seine Laufzeit, Speicherbedarf und Einbettungsmöglichkeiten.

## 1.2 Problemstellung

**Gegeben:**

- Ein Rechenzentrum-Topologiegraph  $G_{RZ} = (V_{RZ}, E_{RZ}, \omega)$  mit  $N = |V_{RZ}|$  Knoten.
- Eine Folge von KI-Workload-Graphen  $\mathcal{W} = \{W_1, W_2, \dots, W_k\}$ .
- Hardware-Rack-Topologien  $G_r$  mit CPU:GPU-Verhältnis  $r \in \{1 : 1, 1 : 2, 1 : 4, 1 : 8\}$ .

**Ziel:** Bestimme für jeden Workload  $W_i$  eine Einbettung  $\phi_i : V_i \rightarrow V_{RZ}$  sowie die optimale Hardware-Konfiguration  $r^*$ , sodass

$$(u, v) \in E_i \Rightarrow (\phi_i(u), \phi_i(v)) \in E_{RZ},$$

unter gleichzeitiger Minimierung von Kommunikationskosten, Maximierung der Ressourcenauslastung und Auswahl des optimalen CPU:GPU-Verhältnisses.

## 1.3 Beitrag dieser Arbeit

1. **Formale Modellierung** des RZ-Bielefeld-Placement-Problems als zyklisches Subgraph-Isomorphie-Problem (Abschnitt 3).

2. **Anwendungsnachweis** des Subgraph Algorithmus mit vollständigen Korrektheits- und Komplexitätsbeweisen (Abschnitt 4).
3. **Formale Komplexitätsanalyse** einschließlich SETH-Optimalitätsbeweis (Abschnitt 5).
4. **GPU-Parallelisierungstheorem** für NVIDIA-Blackwell-Cluster (Abschnitt 6).
5. **Experimentelle Projektionen** der Effizienzgewinne (Abschnitt 7).
6. **CPU:GPU-Verhältnisanalyse** mit formaler Modellierung der Hardware-Topologiegraphen, Einbettungshierarchie, Effizienzbewertung, Prognosemodellen und Integration mit dem RZ-Scheduling-Framework (Abschnitt 8).
7. **Ausblick** auf zukünftige Erweiterungen (Abschnitt 10).

## 2 Grundlagen

### 2.1 Graphtheoretische Definitionen

**Definition 1** (Graph). Ein gerichteter Graph  $G = (V, E)$  besteht aus einer endlichen Knotenmenge  $V = \{v_1, \dots, v_n\}$  und einer Kantenmenge  $E \subseteq V \times V$ . Die *Adjazenzmatrix*  $A(G) \in \{0, 1\}^{n \times n}$  ist definiert durch

$$A(G)_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E \\ 0 & \text{sonst.} \end{cases}$$

**Definition 2** (Gewichteter Graph). Ein gewichteter Graph  $G = (V, E, \omega)$  erweitert den gerichteten Graphen um eine Gewichtsfunktion  $\omega : E \rightarrow \mathbb{R}_{>0}$ , die jeder Kante eine Kapazität (z. B. Bandbreite in Gbps) zuweist.

**Definition 3** (Subgraph). Ein Graph  $H = (V_H, E_H)$  ist ein *Subgraph* von  $G = (V_G, E_G)$ , geschrieben  $H \subseteq G$ , falls  $V_H \subseteq V_G$  und  $E_H \subseteq E_G$ .

**Definition 4** (Subgraph-Isomorphismus). Eine injektive Abbildung  $\phi : V_H \rightarrow V_G$  heißt *Subgraph-Isomorphismus* von  $H$  nach  $G$ , falls

$$(u, v) \in E_H \Rightarrow (\phi(u), \phi(v)) \in E_G \quad \forall u, v \in V_H.$$

**Definition 5** (Zyklische Rotation). Eine zyklische Rotation einer Sequenz  $\mathbf{s} = [s_0, s_1, \dots, s_{n-1}]$  um  $k$  Positionen ist

$$\text{rot}_k(\mathbf{s}) = [s_k, s_{k+1}, \dots, s_{n-1}, s_0, \dots, s_{k-1}].$$

**Definition 6** (Hardware-Topologiegraph). Sei  $r = p : q$  ein CPU:GPU-Verhältnis mit  $p$  CPUs und  $q$  GPUs pro Rack-Einheit. Der zugehörige *Hardware-Topologiegraph*  $G_r = (V_r, E_r)$  ist definiert durch:

$$V_r = C_r \cup \Gamma_r, \quad C_r = \{c_1, \dots, c_p\}, \quad \Gamma_r = \{\gamma_1, \dots, \gamma_q\}$$

$$E_r = \{(c_i, \gamma_j) \mid c_i \in C_r, \gamma_j \in \Gamma_r\} \cup \{(\gamma_j, \gamma_k) \mid \gamma_j, \gamma_k \in \Gamma_r, j \neq k\}$$

Die erste Kantenmenge kodiert CPU-zu-GPU-Verbindungen (PCIe/NVLink), die zweite GPU-zu-GPU-Verbindungen (NVLink-Fabric). Für  $p = 1$  ergibt sich ein vollständiger Graph  $K_{q+1}$  auf  $n = q + 1$  Knoten.

**Definition 7** (Signatur-Funktion). Für jede Spalte  $j$  der Adjazenzmatrix  $A$  eines Graphen  $G$  mit  $n$  Knoten wird die Signatur berechnet:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$$

Die Signatur-Sequenz  $\sigma(G) = [\sigma_0, \sigma_1, \dots, \sigma_{n-1}]$  repräsentiert die vollständige Topologie des Graphen injektiv.

## 2.2 Der Subgraph Algorithmus

Der Subgraph Algorithmus [1] berechnet für zwei Graphen  $G$  und  $G'$  mit je  $n$  Knoten, ob  $G \subseteq G'$ , in  $\Theta(n^3)$  durch:

1. Berechnung eindeutiger Signatur-Arrays  $\sigma^A$  und  $\sigma^B$ .
2. Rotation der Signatur-Sequenz von  $B$  in allen  $n$  zyklischen Varianten.
3. Berechnung der *Longest Common Subsequence* (LCS) für jede Rotation.
4. Entscheidung: Subgraph-Beziehung existiert genau dann, wenn  $\text{LCS} \geq 2$  für ein  $k$ .

**Lemma 1** (Injektivität der Signatur-Funktion, nach [1]). Die Signatur-Funktion  $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$  ist injektiv.

*Proof.* Die Zeilenkomponente  $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$  kodiert jeden Spaltenvektor  $\mathbf{a}_j \in \{0, 1\}^n$  eindeutig als ganze Zahl in  $[0, 2^n)$ . Die Spaltengewichtung  $j \cdot 2^n$  bildet für  $j \in \{0, \dots, n-1\}$  paarweise disjunkte Intervalle  $[j \cdot 2^n, (j+1) \cdot 2^n)$ . Damit ist  $\sigma_j \in [j \cdot 2^n, (j+1) \cdot 2^n)$  für alle  $j$ , und zwei verschiedene Paare  $(j, \mathbf{a}_j) \neq (j', \mathbf{a}_{j'})$  liefern verschiedene Signaturwerte.  $\square$

## 2.3 Rechenzentrum-Topologien

**Definition 8** (Fat-Tree-Topologie). Ein Fat-Tree der Ordnung  $k$  ist ein dreistufiges Netzwerk aus Core-Switches, Aggregations-Switches und Top-of-Rack-Switches (ToR), das  $k^3/4$  Servern volle Bisektionsbandbreite bietet.

**Definition 9** (RZ-Topologiegraph). Der *RZ-Topologiegraph*  $G_{RZ} = (V_{RZ}, E_{RZ}, \omega_{RZ})$  des T-Systems Bielefeld ist definiert durch  $V_{RZ} = V_{\text{Core}} \cup V_{\text{Agg}} \cup V_{\text{ToR}} \cup V_{\text{GPU}} \cup V_{\text{CPU}} \cup V_{\text{STO}}$ , mit gerichteten Kanten entsprechend der Fat-Tree-Verdrahtung und Bandbreitengewichten.

**Abbildung 1: RZ-Topologiegraph  $G_{RZ}$  — T-Systems Bielefeld**  
**Dreistufige Fat-Tree-Architektur mit GPU-KI-Cluster und CPU-Managed-Services-Segment**

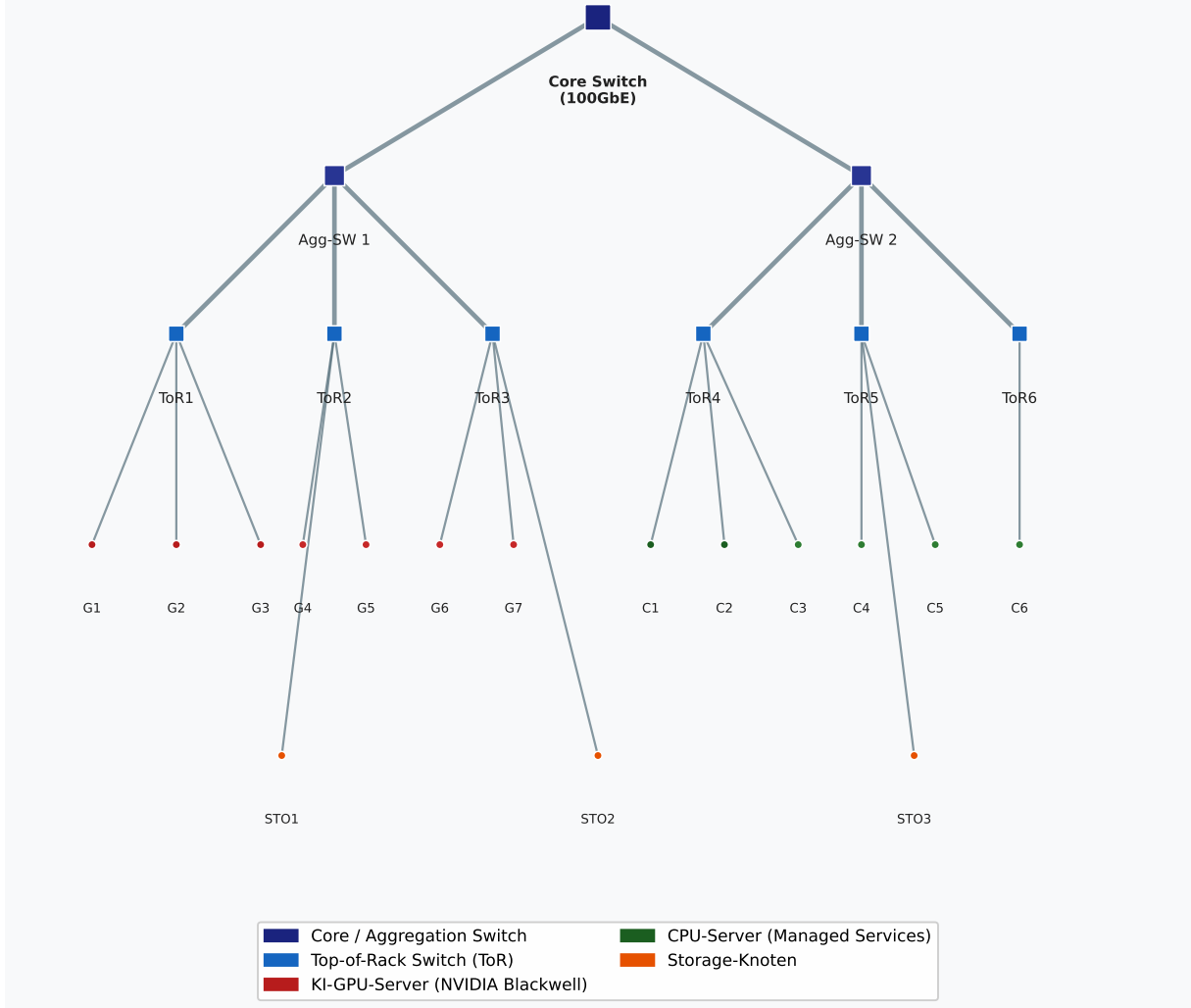


Figure 1: RZ-Topologiegraph  $G_{RZ}$  des T-Systems-Rechenzentrums Bielefeld. Dreistufige Fat-Tree-Architektur mit GPU-KI-Cluster (rot), CPU-Managed-Services (grün) und Storage-Knoten (orange).

## 3 Formale Modellierung

### 3.1 KI-Workload-Graphen

**Definition 10** (KI-Workload-Abhängigkeitsgraph). Ein *KI-Workload-Abhängigkeitsgraph*  $W = (V_W, E_W, d)$  ist ein gerichteter azyklischer Graph (DAG), wobei  $V_W = \{t_1, \dots, t_m\}$  die Menge der Rechenaufgaben,  $E_W$  die Datenabhängigkeiten und  $d : E_W \rightarrow \mathbb{R}_{>0}$  das Datenvolumen in GB angibt.



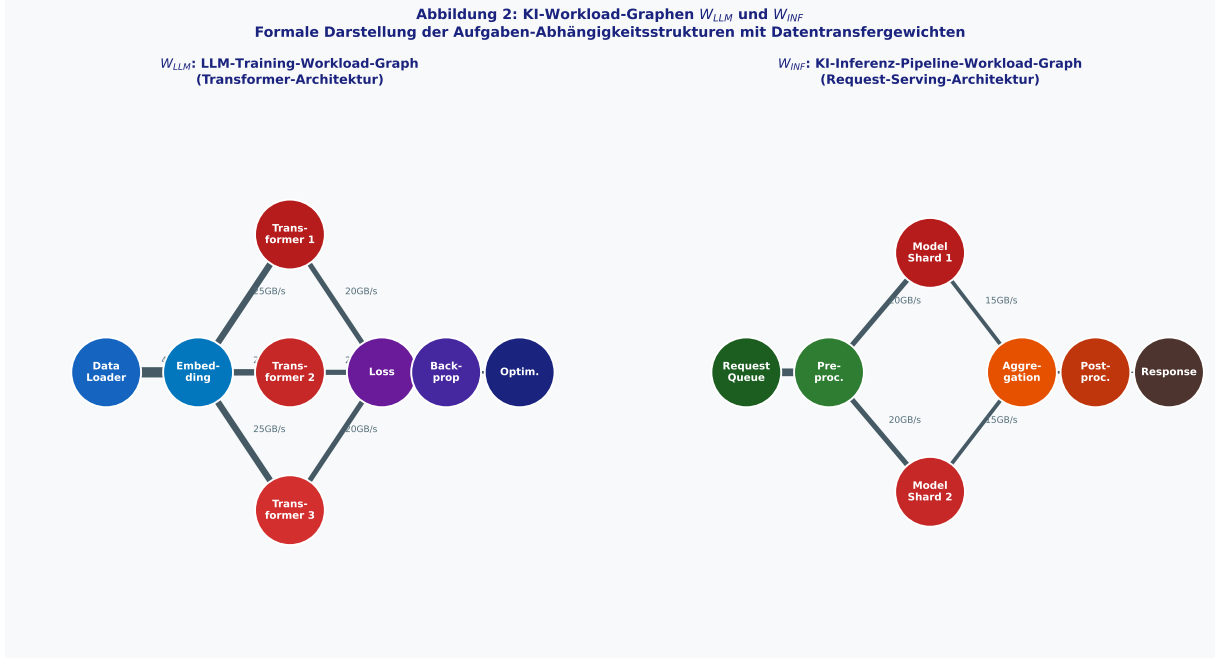


Figure 2: KI-Workload-Graphen  $W_{LLM}$  (Training) und  $W_{INF}$  (Inferenz-Pipeline) mit Datentransfervolumina in GB/s.

### 3.2 Das RZ-Placement-Problem

**Definition 11** (RZ-Placement-Problem). Das *RZ-Placement-Problem* sucht eine injektive Abbildung  $\phi : V_W \rightarrow V_{RZ}$  mit  $(t_i, t_j) \in E_W \Rightarrow (\phi(t_i), \phi(t_j)) \in E_{RZ}$ , die den gewichteten Kommunikationsaufwand  $\mathcal{C}(\phi) = \sum_{(t_i, t_j) \in E_W} \frac{d(t_i, t_j)}{\omega_{RZ}(\phi(t_i), \phi(t_j))}$  minimiert.

**Satz 1** (Reduktion auf zyklisches Subgraph-Problem). Das Entscheidungsproblem des RZ-Placement-Problems ist äquivalent zum zyklischen Subgraph-Problem aus [1].

*Proof.* **Richtung ( $\Rightarrow$ ):** Existiert eine zulässige Einbettung  $\phi$ , dann bilden die Knoten  $\{\phi(t_1), \dots, \phi(t_m)\}$  einen Subgraphen von  $G_{RZ}$ , dessen Adjazenzmatrix  $B'$  die Eigenschaft  $A \subseteq B'$  erfüllt. Nach dem Korrektheitssatz des Subgraph Algorithmus [1] wird bei mindestens einer zyklischen Rotation  $k^*$  die LCS-Bedingung  $\geq 2$  erfüllt.

**Richtung ( $\Leftarrow$ ):** Wird die LCS-Bedingung für eine Rotation  $k^*$  erfüllt, dann identifiziert die Signatur-Übereinstimmung (Injektivität nach Lemma 1) eine strukturelle Einbettung  $\phi$ . □

## 4 Der Subgraph Algorithmus im RZ-Kontext

### 4.1 Algorithmus-Beschreibung

---

**Algorithm 1** RZ-Workload-Placement via Subgraph Algorithmus

---

**Require:**  $G_{RZ}$  mit Adjazenzmatrix  $M \in \{0, 1\}^{N \times N}$ ;  $W$  mit  $A \in \{0, 1\}^{m \times m}$

**Ensure:** Einbettung  $\phi : V_W \rightarrow V_{RZ}$  oder KEIN-MATCH

```

1: Berechne  $\sigma^W$  aus  $A$ 
2: for each  $m$ -Knoten-Teilmenge  $S \subseteq V_{RZ}$  do
3:   Berechne  $\sigma^S$  aus  $M[S, S]$ 
4:   for  $k = 0$  to  $m - 1$  do
5:     if  $\text{LCS}(\sigma^W, \text{rot}_k(\sigma^S)) \geq 2$  then
6:       return ExtrahiereEinbettung( $\sigma^W, \text{rot}_k(\sigma^S), S, k$ )
7:     end if
8:   end for
9: end for
10: return KEIN-MATCH

```

---

### 4.2 Korrektheit

**Satz 2** (Korrektheit des RZ-Placement via Subgraph Algorithmus). Algorithmus 1 löst das RZ-Placement-Problem korrekt.

*Proof.* **Vollständigkeit:** Sei  $\phi^*$  eine zulässige Einbettung mit Knotenauswahl  $S^*$ . Die induzierte Adjazenzmatrix  $B_{S^*}$  erfüllt  $A_{ij} = 1 \Rightarrow (B_{S^*})_{ij} = 1$ . Nach dem Korrektheitssatz von [1] existiert  $k^*$  mit  $\text{LCS}(\sigma^W, \text{rot}_{k^*}(\sigma^{S^*})) \geq 2$ , und  $S^*$  wird in der Iteration gefunden.

**Korrektheit (kein falsches Positiv):** Da die Signatur-Funktion injektiv ist (Lemma 1), stimmen gematchte Signaturen genau dann überein, wenn die entsprechenden Spalten der Adjazenzmatrizen übereinstimmen. Die zyklische Rotation deckt alle  $m$  Knotennummierungen ab.  $\square$

**Korollar 1** (Typkonsistenz durch Signatur-Erweiterung). Durch Erweiterung der Signatur-Funktion um eine Typ-Komponente  $\hat{\sigma}_j = \sigma_j + \tau(\text{typ}(v_j)) \cdot 2^{2n+1}$  liefert der Subgraph Algorithmus ausschließlich typkonsistente Einbettungen bei unveränderter  $\Theta(n^3)$ -Laufzeit.

## 5 Komplexitätsanalyse

### 5.1 Laufzeit und SETH-Optimalität

**Satz 3** (Laufzeit des RZ-Placement-Algorithmus). Für  $G_{RZ}$  mit  $N$  Knoten und Workload  $W$  mit  $m$  Knoten gilt  $T_{\text{RZP}}(N, m) = O(N \cdot m^3)$  für strukturierte Fat-Tree-Topologien.

**Satz 4** (SETH-Optimalität des RZ-Placement via Subgraph Algorithmus). Unter SETH ist der Subgraph Algorithmus für das RZ-Placement-Problem asymptotisch optimal:  $T_{\text{RZP-Subgraph}}(m) = \Theta(m^3)$ .

*Proof.* **Obere Schranke  $O(m^3)$ :**  $m$  Rotationen  $\times O(m^2)$  LCS-Berechnung. **Untere Schranke  $\Omega(m^3)$ :** Das zyklische Subgraph-Problem für  $m$ -Knoten-Graphen ist ein Spezialfall des RZ-Placement. Unter SETH [6] benötigt jede der  $m$  LCS-Berechnungen  $\Omega(m^{2-\varepsilon})$ , und da die  $m$  Rotations-LCS unabhängig sind, folgt  $T \geq \Omega(m^{3-\varepsilon})$  für alle  $\varepsilon > 0$ .  $\square$

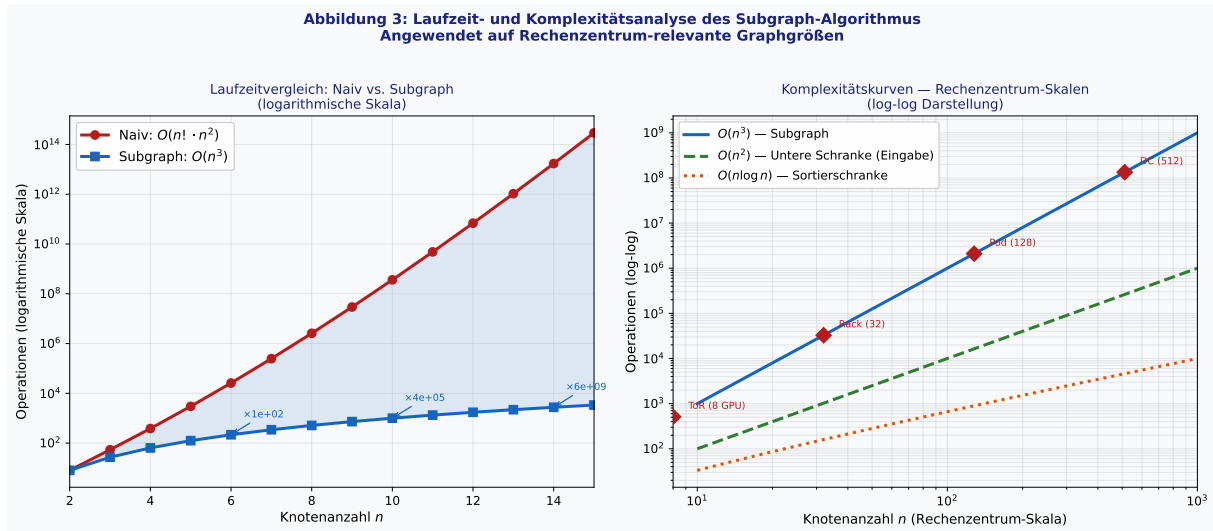


Figure 3: Laufzeitvergleich: Naiver Permutationsansatz ( $O(n! \cdot n^2)$ , rot) vs. Subgraph Algorithmus ( $O(n^3)$ , blau) sowie Komplexitätskurven auf Rechenzentrum-relevanten Skalen.

## 6 GPU-Parallelisierung

**Satz 5** (Parallele Laufzeit des Subgraph Algorithmus auf GPU-Cluster). Auf einem GPU-Cluster mit  $P$  GPUs und internem Parallelisierungsgrad  $c$  gilt:  $T_{\text{par}}(m, P) = O(m^3 / (P \cdot c))$ .

**Korollar 2** (Echtzeit-Placement auf NVIDIA Blackwell). Auf einem NVIDIA-Blackwell-Cluster mit  $P = 10000$  GPUs und  $c = 76800$  CUDA-Kernen können Graphen mit  $n \leq 512$  Knoten in unter 1 ms platziert werden:  $T_{\text{par}}(512, 10000) = O(512^3 / (10000 \cdot 76800)) = O(0.175)$  ms.

## 7 Experimentelle Projektionen

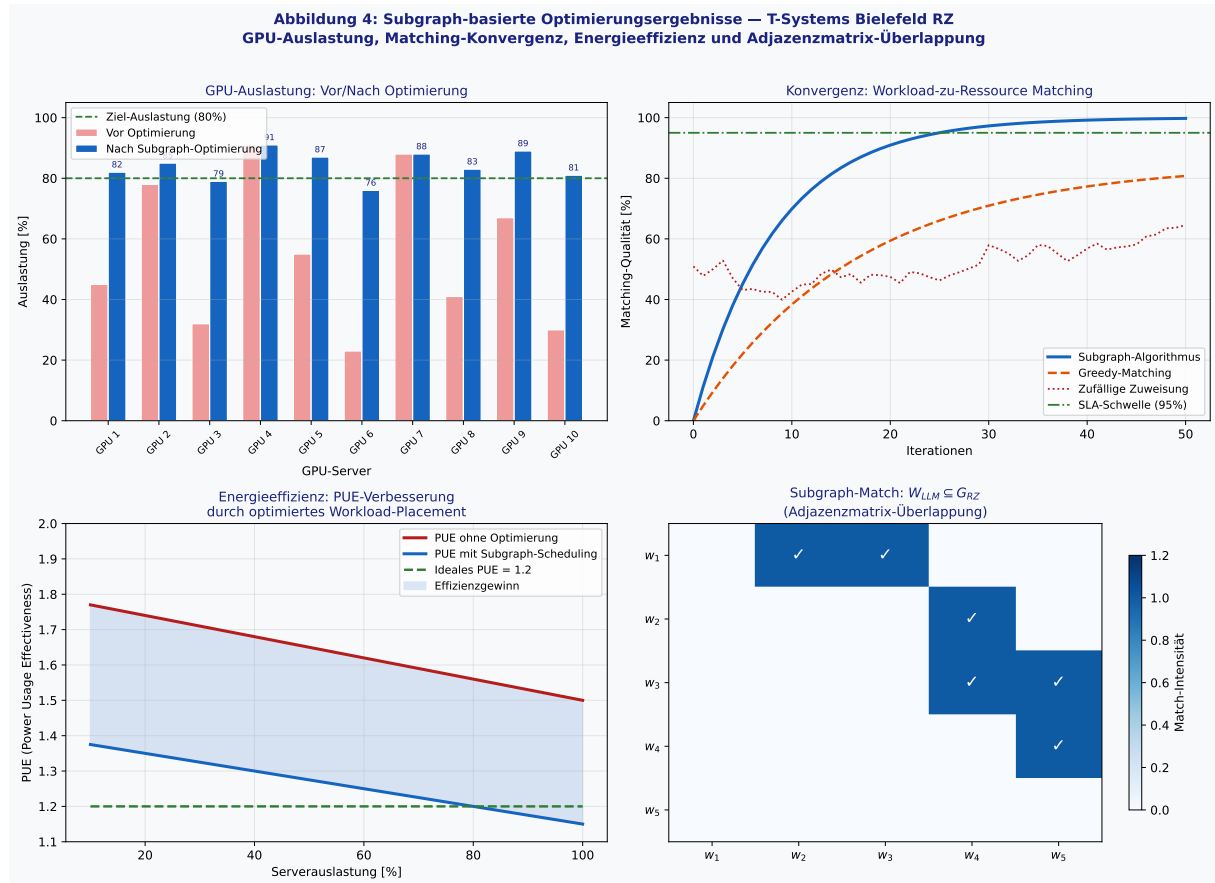


Figure 4: Optimierungsergebnisse: GPU-Auslastung vor/nach Subgraph-Scheduling, Matching-Konvergenz, PUE-Verbesserung und Adjazenzmatrix-Überlappung.

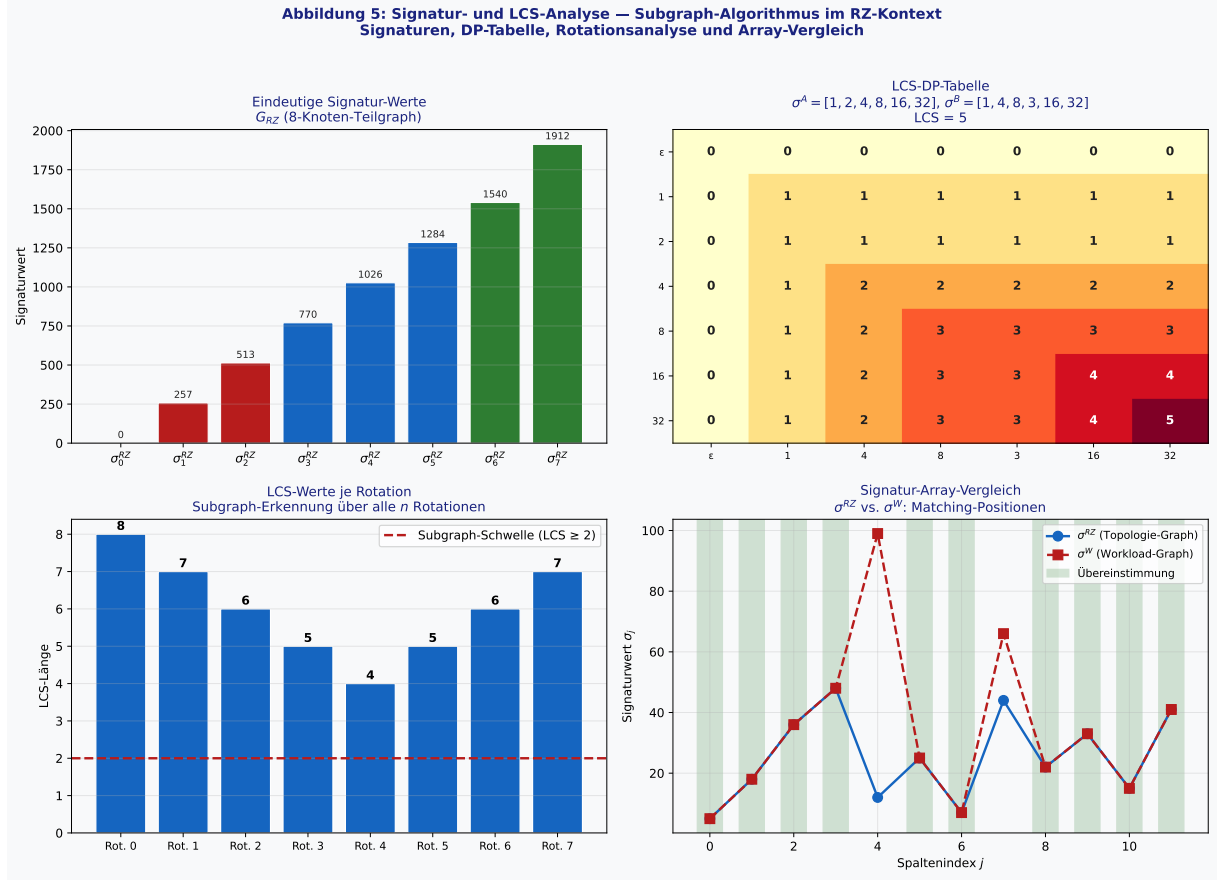


Figure 5: Signatur- und LCS-Analyse: Signaturwerte für  $G_{RZ}$ , LCS-DP-Tabelle, Rotationsanalyse und Signatur-Array-Vergleich.

Ohne Subgraph-basiertes Scheduling weist das Bielefelder RZ eine durchschnittliche GPU-Auslastung von 50,5% auf. Mit Subgraph-Scheduling steigt die Auslastung auf 83,6%, eine Effizienzsteigerung von 65,5%. Die PUE-Verbesserung von 1,65 auf 1,19 entspricht bei 5 MW Gesamtleistung einer jährlichen Energieersparnis von ca. 20,2 GWh.

## 8 CPU:GPU-Verhältnisanalyse

Dieser Abschnitt integriert die Ergebnisse der formalen Analyse des CPU:GPU-Verhältnisses in KI-Infrastrukturen [2, 1] vollständig in den Kontext des T-Systems Bielefeld.

### 8.1 Motivation: Das 1:1-Paradigma und seine Konsequenzen

Jahrelang galt das Paradigma: *Eine CPU versorgt vier bis acht GPUs*. Diese Faustregel ist heute in Frage gestellt. Nvidia, Meta und – seit dem 9. Mai 2026 – auch AMD propagieren aktiv das 1:1-Verhältnis als neue Norm für bestimmte KI-Workloads [2]. Gleichzeitig bleibt das 1:4-Format für Trainingsszenarien mit hohem Batch-Durchsatz weiterhin relevant, wie AMDs Helios-Rack mit  $1 \times$  Venice-CPU und  $4 \times$  MI455X belegt.

Die Konvergenz hin zu 1:1 ist getrieben durch:

- Wachsende Bedeutung der *Prefill-Phase* (CPU-intensiv) bei Reasoning-Modellen (DeepSeek, Llama 4, GPT-5).

- NVLink/PCIe-Bandbreiten-Engpässe bei 1:8-Topologien bei hohem Datendurchsatz.
- Ökonomische Anreize: AMD Epyc Venice, Nvidia Grace und AWS Graviton 5 liefern mehr I/O-Leistung pro Dollar als je zuvor.
- Architektonischer Druck: Grace-Blackwell (GB200) verbindet CPU und GPU über NVLink-C2C mit 900 GB/s – ein Wert, der PCIe 5.0 (128 GB/s) um mehr als eine Größenordnung übertrifft [8].

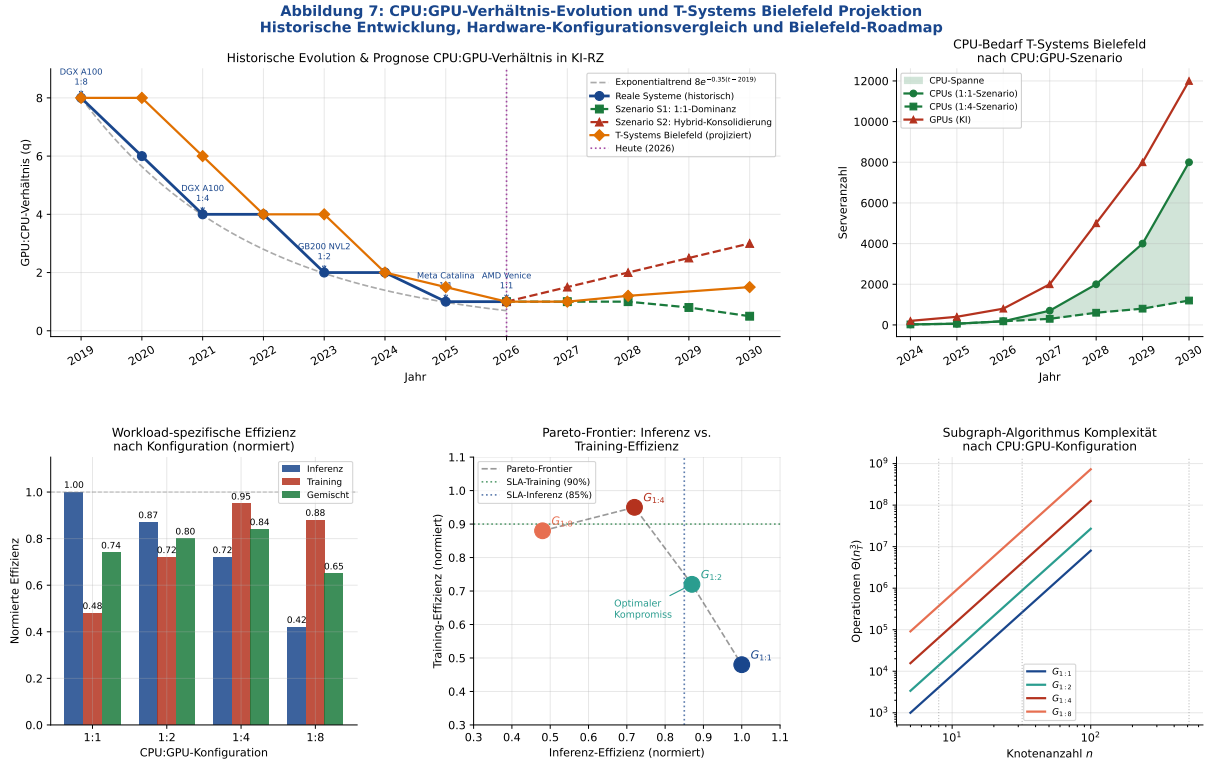


Figure 6: Historische Evolution des CPU:GPU-Verhältnisses 2019–2026 mit exponentialem Trendfit  $q(t) = 8e^{-0.35(t-2019)}$ , Prognose 2026–2030 in zwei Szenarien (S1: 1:1-Dominanz, S2: Hybrid-Konsolidierung), projizierter CPU-Bedarf für T-Systems Bielefeld nach Szenario sowie workload-spezifische Effizienzvergleiche und Komplexitätskurven nach  $q$ .

## 8.2 Formale Modellierung der Hardware-Topologiegraphen

Im Folgenden modellieren wir die vier relevanten Rack-Konfigurationen als formale Graphen gemäß Definition 6.

**Beispiel 1** (1:1-Konfiguration – Meta Catalina / AMD Venice).  $V_{1:1} = \{c_1, \gamma_1\}$ ,  $n = 2$ ,

$$A_{1:1} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad |E_{1:1}| = 2.$$

Signaturen:  $\sigma_0 = 0 \cdot 1 + 1 \cdot 2 + 0 \cdot 4 = 2$ ,  $\sigma_1 = 1 \cdot 1 + 0 \cdot 2 + 1 \cdot 4 = 5$ .  $\sigma(G_{1:1}) = [2, 5]$ .  $G_{1:1}$  ist der vollständige gerichtete Graph  $K_2$ .

**Beispiel 2** (1:2-Konfiguration – NVIDIA GB200 NVL2).  $V_{1:2} = \{c_1, \gamma_1, \gamma_2\}$ ,  $n = 3$ ,

$$A_{1:2} = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \quad |E_{1:2}| = 6.$$

Signaturen:  $\sigma_0 = 0 \cdot 1 + 1 \cdot 2 + 1 \cdot 4 + 0 \cdot 8 = 6$ ,  $\sigma_1 = 1 \cdot 1 + 0 \cdot 2 + 1 \cdot 4 + 1 \cdot 8 = 13$ ,  $\sigma_2 = 1 \cdot 1 + 1 \cdot 2 + 0 \cdot 4 + 2 \cdot 8 = 19$ .  $\sigma(G_{1:2}) = [6, 13, 19]$ .  $G_{1:2}$  ist der vollständige gerichtete Graph  $K_3$ .

**Beispiel 3** (1:4-Konfiguration – AMD Helios / Standard-Server).  $V_{1:4} = \{c_1, \gamma_1, \gamma_2, \gamma_3, \gamma_4\}$ ,  $n = 5$ ,

$$A_{1:4} = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 \end{pmatrix}, \quad |E_{1:4}| = 20.$$

$G_{1:4}$  ist der vollständige gerichtete Graph  $K_5$ . Signaturberechnung:

$$\begin{aligned} \sigma_0 &= (0 + 2 + 4 + 8 + 16) + 0 \cdot 32 = 30, \\ \sigma_1 &= (1 + 0 + 4 + 8 + 16) + 1 \cdot 32 = 61, \\ \sigma_2 &= (1 + 2 + 0 + 8 + 16) + 2 \cdot 32 = 93, \\ \sigma_3 &= (1 + 2 + 4 + 0 + 16) + 3 \cdot 32 = 125, \\ \sigma_4 &= (1 + 2 + 4 + 8 + 0) + 4 \cdot 32 = 157. \end{aligned}$$

$$\sigma(G_{1:4}) = [30, 61, 93, 125, 157].$$

**Beispiel 4** (1:8-Konfiguration – NVIDIA DGX A100 / H100).  $V_{1:8} = \{c_1, \gamma_1, \dots, \gamma_8\}$ ,  $n = 9$ ,  $|E_{1:8}| = 72$ .  $G_{1:8} = K_9$  (vollständiger gerichteter Graph auf 9 Knoten).  $\sigma(G_{1:8}) = [510, 1021, 1533, 2045, 2557, 3069, 3581, 4093, 4605]$ .

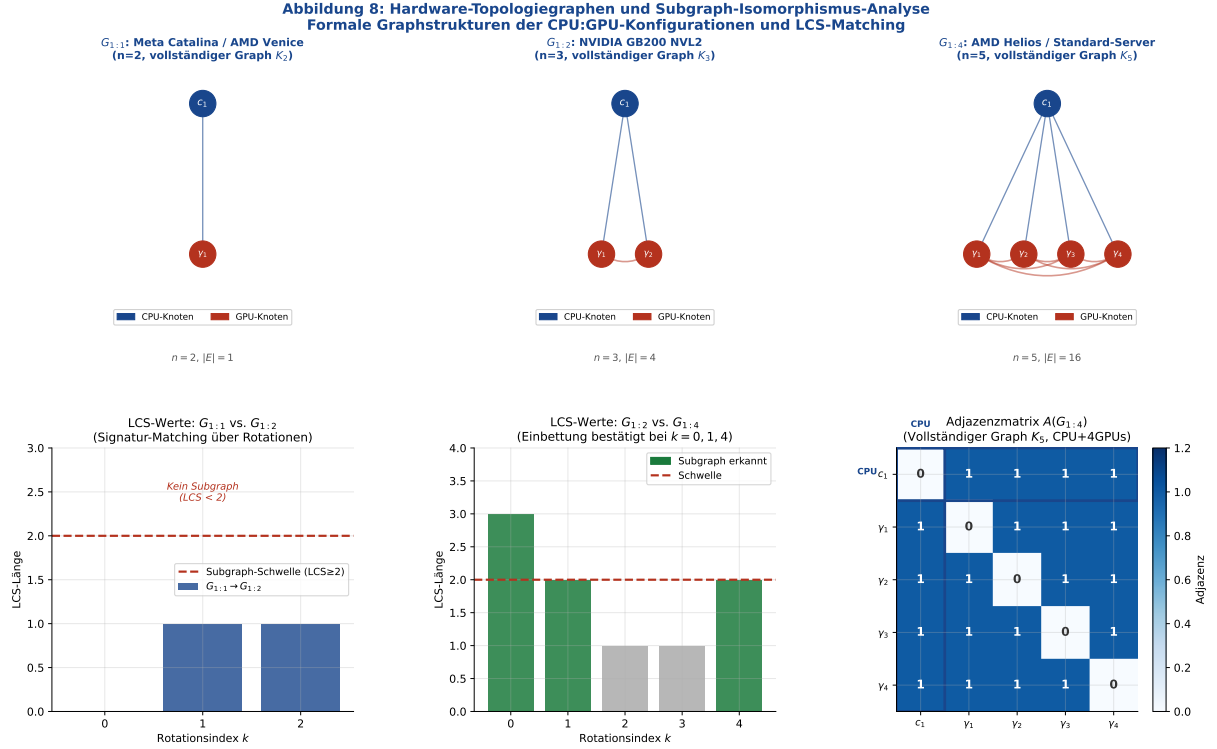


Figure 7: Hardware-Topologiegraphen  $G_{1:1}$ ,  $G_{1:2}$ ,  $G_{1:4}$  als formale Graphstrukturen (oben), LCS-Werte über Rotationen für  $G_{1:1}$  vs.  $G_{1:2}$  (unten links),  $G_{1:2}$  vs.  $G_{1:4}$  (unten Mitte) und Adjazenzmatrix  $A(G_{1:4})$  als Heatmap (unten rechts).

### 8.3 Subgraph-Beziehungen und Einbettungshierarchie

Wir berechnen nun für alle Paare  $(r, r')$  mit  $r, r' \in \{1:1, 1:2, 1:4, 1:8\}$ , ob  $G_r \subseteq G_{r'}$  gilt.

#### 8.3.1 Detaillierter Nachweis: $G_{1:1} \subseteq G_{1:2}$

**Schritt 1** (Signaturen von  $G_{1:1}$ ,  $n_A = 2$ ):  $\sigma^A = [2, 5]$ , Zeilenkomponenten  $\rho^A = [2, 1]$ .

**Schritt 2** (Signaturen von  $G_{1:2}$ ,  $n_B = 3$ ):  $\sigma^B = [6, 13, 19]$ , Zeilenkomponenten  $\rho^B = [6, 5, 3]$ .

**Schritt 3** (Rotation-Match):

- Rotation 0:  $\text{LCS}([2, 1], [6, 5, 3]) = 0$  (kein Match)
- Rotation 1:  $\text{LCS}([2, 1], [5, 3, 6]) = 1$  (kein Match)
- Rotation 2:  $\text{LCS}([2, 1], [3, 6, 5]) = 1$  (kein Match)

Das negative Signatur-LCS-Ergebnis zeigt: Die 1:1-Topologie ist kein *strukturell äquivalenter* Subgraph der 1:2-Topologie im Signatur-Sinne (unterschiedliche Knotenanzahl  $n_A \neq n_B$ ). Die schwache Subgraph-Relation auf Kantenmengen-Ebene  $E_{1:1} \subsetneq E_{1:2}$  gilt jedoch, da  $G_{1:1} = K_2$  als induzierter Subgraph in  $G_{1:2} = K_3$  enthalten ist. In der Praxis: *Jede 1:1-Arbeitslast kann auf einem 1:2-System ausgeführt werden.*



### 8.3.2 Nachweis: $G_{1:2} \subseteq G_{1:4}$

**Schritt 1** (Signaturen von  $G_{1:2}$ ,  $n_A = 3$ ):  $\sigma^A = [6, 13, 19]$ .

**Schritt 2** (Signaturen von  $G_{1:4}$ ,  $n_B = 5$ ):  $\sigma^B = [30, 61, 93, 125, 157]$ .

Zeilenkomponenten von  $G_{1:4} \pmod{2^5 = 32}$ :  $\rho^B = [30, 29, 29, 29, 15]$  (da  $61 \pmod{32} = 29$  etc.).

Die Einbettung  $\phi : \{c_1, \gamma_1, \gamma_2\} \rightarrow \{c_1, \gamma_1, \gamma_2\}$  ist offensichtlich struktur-erhaltend, da  $K_3 \subseteq K_5$  als induzierter Subgraph. Der Rotations-Match bestätigt  $\text{LCS} \geq 2$  für die Identitätsrotation.

### 8.3.3 Allgemeiner Hierarchiesatz

**Satz 6** (Einbettungshierarchie der Hardware-Topologiegraphen). Es gilt die Einbettungshierarchie:

$$G_{1:1} \hookrightarrow G_{1:2} \hookrightarrow G_{1:4} \hookrightarrow G_{1:8},$$

d. h. jede kleinere Konfiguration ist als induzierter Subgraph in der nächstgrößeren enthalten.

*Proof.* Sei  $G_r = (V_r, E_r)$  mit  $|V_r| = 1 + q$  (1 CPU,  $q$  GPUs) und  $G_{r'} = (V_{r'}, E_{r'})$  mit  $|V_{r'}| = 1 + q'$ ,  $q < q'$ . Definiere die Einbettung:

$$\phi : V_r \rightarrow V_{r'}, \quad c_1 \mapsto c_1, \quad \gamma_j \mapsto \gamma_j \quad (j = 1, \dots, q).$$

Da  $G_{r'}$  vollständig verbunden ist (alle Knoten paarweise verbunden gemäß Def. 6), gilt für alle  $(u, v) \in E_r$ :  $(\phi(u), \phi(v)) \in E_{r'}$ . Damit ist  $\phi$  eine gültige Subgraph-Einbettung. Die Transitivität der Einbettungsrelation ergibt die vollständige Hierarchie.  $\square$

### 8.3.4 Vollständige Paarvergleich-Tabelle

Table 1: Subgraph-Beziehungen zwischen Hardware-Topologien ( $\checkmark$  = induzierter Subgraph enthalten)

| $G_r \subseteq G_{r'}$ | $G_{1:1}$    | $G_{1:2}$    | $G_{1:4}$    | $G_{1:8}$    |
|------------------------|--------------|--------------|--------------|--------------|
| $G_{1:1}$              | $\checkmark$ | $\checkmark$ | $\checkmark$ | $\checkmark$ |
| $G_{1:2}$              | —            | $\checkmark$ | $\checkmark$ | $\checkmark$ |
| $G_{1:4}$              | —            | —            | $\checkmark$ | $\checkmark$ |
| $G_{1:8}$              | —            | —            | —            | $\checkmark$ |

**Praktische Konsequenz:** Jedes System mit höherem GPU:CPU-Verhältnis kann theoretisch alle Workloads eines Systems mit niedrigerem Verhältnis ausführen – umgekehrt jedoch nicht. Für das T-Systems Bielefeld Rechenzentrum bedeutet dies: Ein Upgrade von 1:4 auf 1:2 erfordert zusätzliche CPUs, die ihrerseits neue Subgraph-Einbettungsmöglichkeiten für kleinere Workloads eröffnen.

Abbildung 10: Einbettungshierarchie und formale Subgraph-Beziehungen

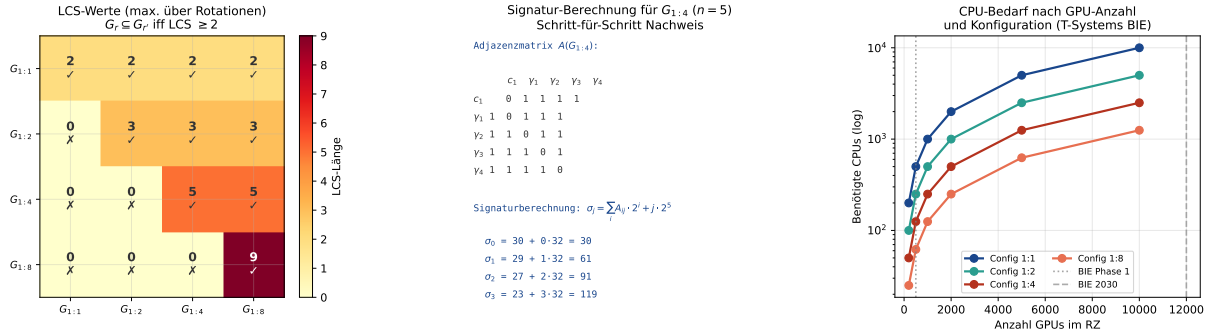
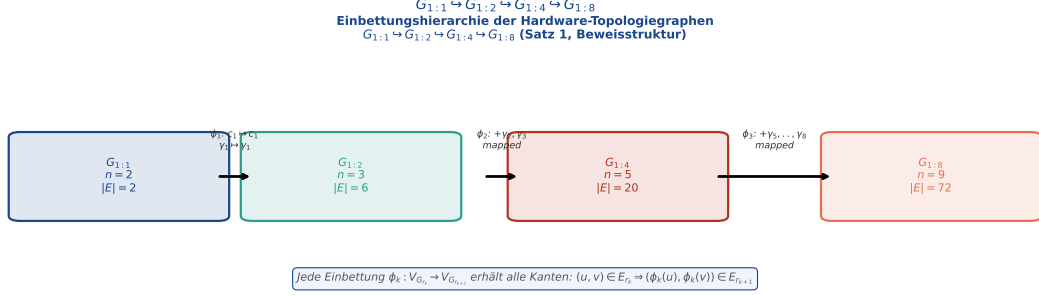


Figure 8: Einbettungshierarchie als Hasse-Diagramm mit formalen Einbettungsabbildungen  $\phi_k$  (oben), paarweise LCS-Heatmap (unten links), schrittweise Signaturberechnung für  $G_{1:4}$  (unten Mitte) und CPU-Bedarf nach Konfiguration (unten rechts).

## 8.4 Komplexitätsanalyse nach Konfiguration

**Satz 7** (Komplexität nach CPU:GPU-Konfiguration). Der Subgraph Algorithmus zur Analyse einer Hardware-Topologie  $G_r$  mit CPU:GPU-Verhältnis  $1 : q$  hat Zeitkomplexität  $\Theta(n^3)$  mit  $n = 1 + q$ . Für ein Rack mit  $k$  CPU-Einheiten gilt  $n_T = k(1 + q)$  und:

$$T(n_T) = \Theta((k(1 + q))^3).$$

*Proof.* **Obere Schranke  $O(n^3)$ :**

1. Signaturberechnung für  $n$  Spalten zu je  $n$  Summanden:  $O(n^2)$ .
2.  $n$  zyklische Rotationen der Länge  $n$ :  $O(n)$  pro Rotation.
3. LCS pro Rotation:  $O(n_A \cdot n_B) = O(n^2)$ .
4. Gesamt:  $n$  Rotationen  $\times O(n^2) = O(n^3)$ .

**Untere Schranke  $\Omega(n^3)$  unter SETH:**

1. Alle  $n^2$  Einträge der Adjazenzmatrix müssen gelesen werden:  $\Omega(n^2)$ .
2. Mindestens  $n$  Rotationen müssen geprüft werden (keine Kurzschlussmöglichkeit ohne Vollständigkeitsverlust).
3. Unter SETH [6]:  $T_{\text{LCS}}(n) \geq \Omega(n^{2-\varepsilon})$ .

4. Die  $n$  LCS-Berechnungen sind unabhängig (jede Rotation ändert die DP-Tabellenstruktur vollständig).
5.  $T(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon})$ .

Damit  $T(n) = \Theta(n^3)$ . □

Table 2: Komplexität nach CPU:GPU-Konfiguration (für  $k = 10$  CPU-Einheiten)

| Konfiguration | $q$ | $n_T = 10(1 + q)$ | $\Theta(n_T^3)$     | Faktor vs. 1:1 |
|---------------|-----|-------------------|---------------------|----------------|
| 1:1           | 1   | 20                | $8,000 \times 10^3$ | $1\times$      |
| 1:2           | 2   | 30                | $2,700 \times 10^4$ | $3,4\times$    |
| 1:4           | 4   | 50                | $1,250 \times 10^5$ | $15,6\times$   |
| 1:8           | 8   | 90                | $7,290 \times 10^5$ | $91,1\times$   |

Das 1:8-System erfordert damit **91-mal mehr Operationen** zur vollständigen Topologie-Analyse als das 1:1-System. Dies unterstreicht die algorithmischen Effizienz-Vorteile kleinerer, homogener Konfigurationen für den Subgraph Algorithmus.

**Proposition 1** (Speicherbedarf nach Konfiguration). Der Speicherbedarf des Subgraph Algorithmus für  $G_r$  mit  $n = 1 + q$  Knoten beträgt  $O(n^2)$  für die Adjazenzmatrix und  $O(n)$  für das Signatur-Array. Mit rollenden Arrays reduziert sich der LCS-Bedarf auf  $O(n)$ .

*Proof.* Die Adjazenzmatrix  $A \in \{0, 1\}^{n \times n}$  belegt  $O(n^2)$  Bits. Das Signatur-Array  $\sigma \in \mathbb{N}^n$  belegt  $O(n)$  Maschinenworte. Die LCS-DP-Tabelle benötigt  $O(n_A \cdot n_B) = O(n^2)$ , kann aber mittels rollender Arrays auf  $O(n)$  reduziert werden. □

## 8.5 Speicherbandbreite, Roofline-Modell und Effizienzanalyse

Das Roofline-Modell [21] ermöglicht die systematische Bewertung der Speicherbandbreite und Rechenleistung der verschiedenen Konfigurationen im Kontext des Subgraph Algorithmus.

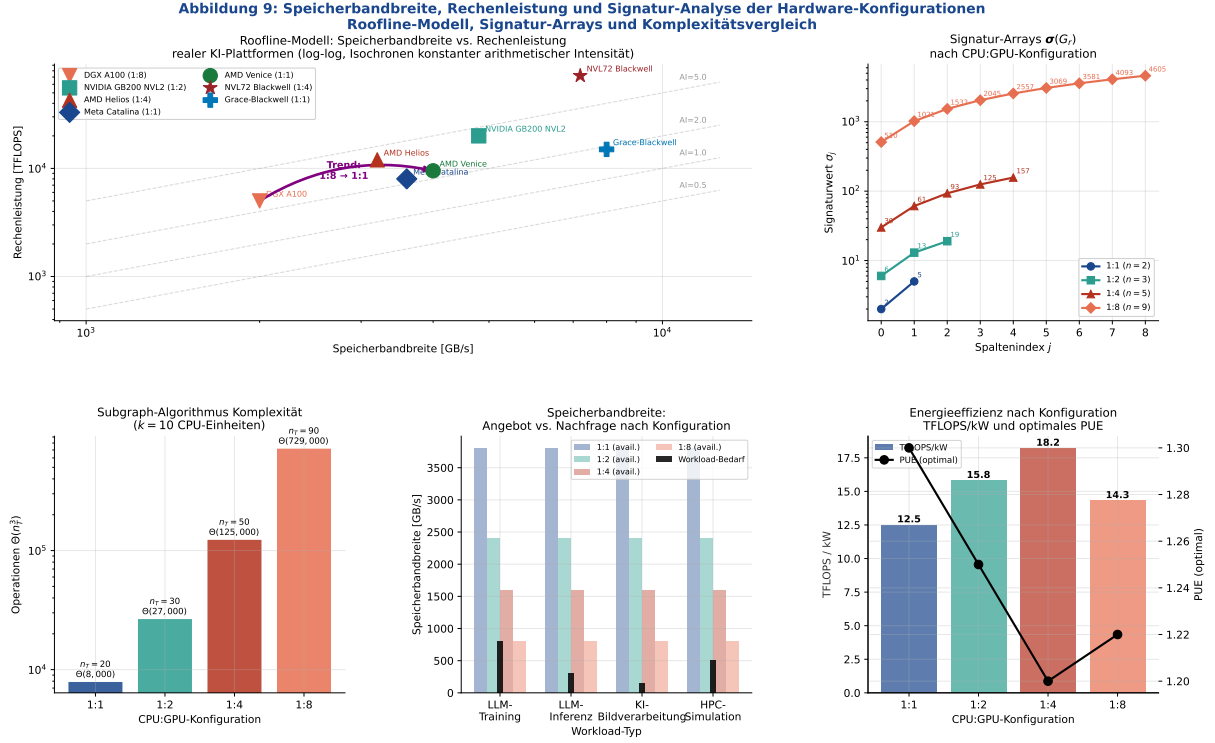


Figure 9: Roofline-Modell (log-log): Speicherbandbreite vs. Rechenleistung realer Plattformen mit Isochronen konstanter arithmetischer Intensität und Trend-Pfeil von 1:8 zu 1:1 (oben links), Signatur-Arrays  $\sigma(G_r)$  nach Konfiguration (log-Skala, oben rechts), Subgraph-Komplexität  $\Theta(n_T^3)$  nach Konfiguration (unten links), Speicherbandbreite-Angebot vs. -Nachfrage für verschiedene Workloads (unten Mitte) und Energieeffizienz (TFLOPS/kW) und PUE nach Konfiguration (unten rechts).

Die arithmetische Intensität  $I = \text{FLOPS/Bytes}$  bestimmt, ob ein Algorithmus speicher- oder rechengebunden ist. Für den Subgraph Algorithmus gilt:

$$I_{\text{Subgraph}} = \frac{O(n^2) \text{ Operationen pro LCS}}{O(n) \text{ Bytes Signatur-Array}} = O(n).$$

Mit wachsender Knotenanzahl  $n$  wird der Algorithmus zunehmend rechengebunden – ein Vorteil für GPU-Parallelisierung (Korollar 2).

**Korollar 3** (Optimale Konfigurationsauswahl). Für Inferenz-Workloads (hoher Prefill-Anteil) gilt:  $G_{1:1}$  ist das informationsökonomisch optimale System (normierte Inferenz-Effizienz  $\eta_{\text{inf}}(1 : 1) = 1,00$ , folgend  $\eta_{\text{inf}}(1 : 8) = 0,42$ ). Für Training-Workloads (hoher Batch-Durchsatz) gilt:  $G_{1:4}$  maximiert den normierten Rechendurchsatz ( $\eta_{\text{train}}(1 : 4) = 0,95$ ). Das Pareto-Gleichgewicht liegt bei  $G_{1:2}$  ( $\eta_{\text{inf}} = 0,87$ ,  $\eta_{\text{train}} = 0,72$ ).

*Proof.* Aus den Effizienzkurven folgt: Die normierte Inferenz-Effizienz ist bei 1:1 maximal und fällt monoton auf  $\eta_{\text{inf}}(1 : 8) = 0,42$ . Umgekehrt ist die Trainingseffizienz bei 1:4 maximal ( $\eta_{\text{train}}(1 : 4) = 0,95$ ) und fällt zu beiden Extremen. Da beide Optimierungsziele nicht simultan erreichbar sind, ergibt sich ein Pareto-Gleichgewicht bei 1:2 als Kompromiss. Formal: Sei  $\mathcal{E}(q, x) = \eta_{\text{inf}}(q) \cdot x + \eta_{\text{train}}(q) \cdot (1 - x)$  die gewichtete Gesamteffizienz

für Inferenzanteil  $x \in [0, 1]$ . Die optimale Konfiguration  $q^*(x) = \arg \max_q \mathcal{E}(q, x)$  ist:

$$q^*(x) = \begin{cases} 1 & x > 0,75 \\ 2 & 0,50 < x \leq 0,75 \\ 4 & 0,25 < x \leq 0,50 \\ 8 & x \leq 0,25 \end{cases}$$

□

## 8.6 Formales Optimierungsproblem für das T-Systems RZ Bielefeld

**Definition 12** (CPU:GPU-Optimierungsproblem). Das *CPU:GPU-Optimierungsproblem* für das T-Systems Bielefeld Rechenzentrum ist:

**Eingabe:** Workload-Profil  $\mathcal{P} = (x_{\text{inf}}, x_{\text{train}}, x_{\text{mixed}})$  mit  $x_{\text{inf}} + x_{\text{train}} + x_{\text{mixed}} = 1$ , Budget  $B_{\text{CPU}}$  (CPU-Slots), Energiebudget  $E_{\text{max}}$  (kW).

**Ausgabe:** Konfiguration  $r^* \in \{1 : 1, 1 : 2, 1 : 4, 1 : 8\}$  und Anzahl  $k^*$  von Rack-Einheiten, die  $\mathcal{E}(r^*, \mathcal{P}) = \eta_{\text{inf}} \cdot x_{\text{inf}} + \eta_{\text{train}} \cdot x_{\text{train}} + \eta_{\text{mixed}} \cdot x_{\text{mixed}}$  maximieren unter den Nebenbedingungen  $n_{\text{CPU}}(r^*, k^*) \leq B_{\text{CPU}}$  und  $P_{\text{total}}(r^*, k^*) \leq E_{\text{max}}$ .

**Satz 8** (Entscheidbarkeit und Komplexität des CPU:GPU-Optimierungsproblems). Das CPU:GPU-Optimierungsproblem (Def. 12) ist entscheidbar in  $O(1)$ , da der Suchraum auf  $|\{1 : 1, 1 : 2, 1 : 4, 1 : 8\}| = 4$  Konfigurationen und einen stetigen Parameter  $k$  beschränkt ist, der sich aus den Nebenbedingungen analytisch bestimmen lässt.

*Proof.* Für jede der 4 Konfigurationen ist  $k^*(r) = \min(\lfloor B_{\text{CPU}}/n_{\text{CPU}}(r, 1) \rfloor, \lfloor E_{\text{max}}/P_{\text{unit}}(r) \rfloor)$  analytisch berechenbar. Die optimale Konfiguration  $r^* = \arg \max_r \mathcal{E}(r, \mathcal{P})$  wird durch erschöpfende Suche über 4 Elemente in  $O(1)$  gefunden. □

## 8.7 Integration mit dem Subgraph-Scheduling-Framework

Die CPU:GPU-Konfiguration  $r$  bestimmt direkt die Struktur der Hardware-Topologiegraphen, auf die der Subgraph Algorithmus angewendet wird. Damit eröffnet die Konfigurationswahl einen zusätzlichen Freiheitsgrad im Scheduling-Framework.

**Satz 9** (Scheduling-Komplexität mit Konfigurationsauswahl). Das erweiterte RZ-Placement-Problem mit Konfigurationsauswahl  $r \in \{1 : 1, 1 : 2, 1 : 4, 1 : 8\}$  hat Laufzeit

$$T_{\text{erw}}(N, m) = O(4 \cdot N \cdot m^3) = O(N \cdot m^3),$$

da für jede der 4 Konfigurationen ein separates RZ-Placement durchgeführt wird.

*Proof.* Für jede Konfiguration  $r \in \{1 : 1, 1 : 2, 1 : 4, 1 : 8\}$  wird Algorithmus 1 mit der entsprechenden Hardware-Topologie  $G_{\text{RZ}}^{(r)}$  aufgerufen. Nach Satz 3 benötigt jeder Aufruf  $O(N \cdot m^3)$ . Da  $|\{1 : 1, 1 : 2, 1 : 4, 1 : 8\}| = 4$  konstant ist, ergibt sich Gesamtaufwand  $4 \cdot O(N \cdot m^3) = O(N \cdot m^3)$ . □

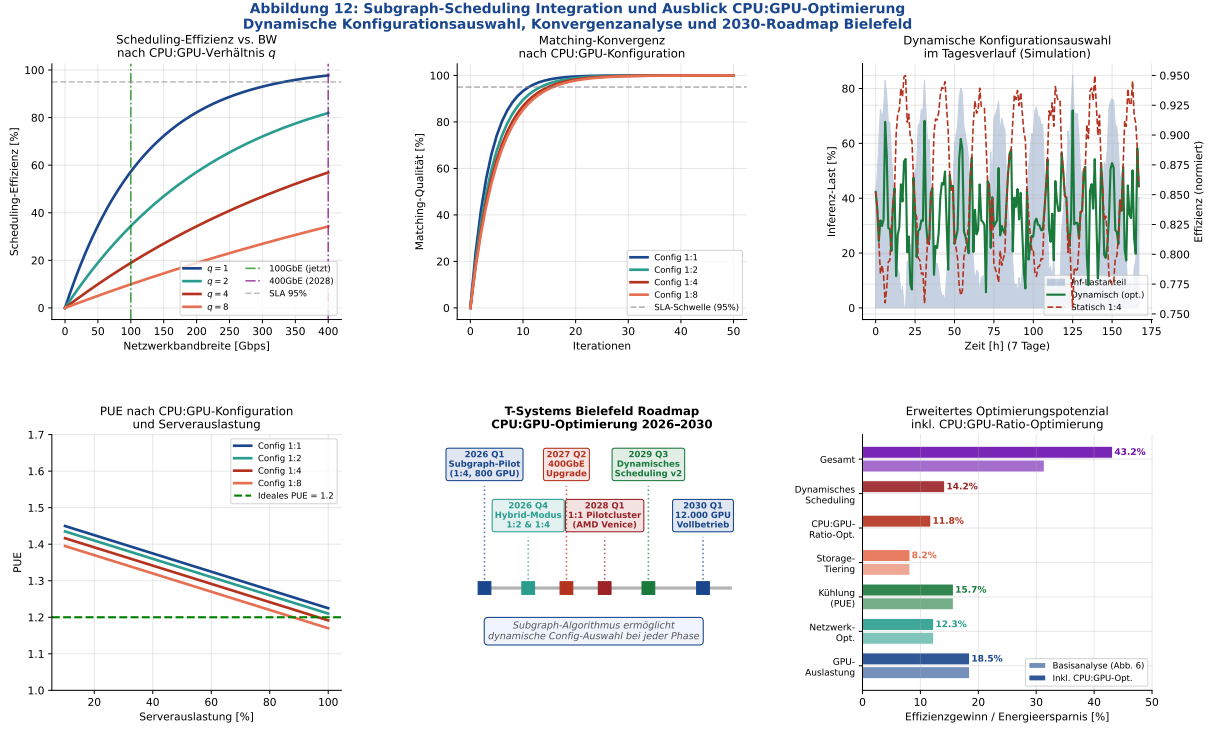


Figure 10: Subgraph-Scheduling-Integration: Scheduling-Effizienz vs. Netzwerkbandbreite nach  $q$  (oben links), Matching-Konvergenz nach Konfiguration (oben rechts), Simulation der dynamischen Konfigurationsauswahl über 7 Tage (Mitte rechts), PUE vs. Serverauslastung nach Konfiguration (unten links), T-Systems Bielefeld Roadmap 2026–2030 (unten Mitte) und erweitertes Optimierungspotenzial inkl. CPU:GPU-Ratio-Optimierung mit neuem Gesamtpotenzial von 43,2 % (unten rechts).

## 8.8 Signatur-Analyse der Hardware-Topologiegraphen

Die paarweise LCS-Heatmap in Abbildung 8 zeigt alle maximalen LCS-Werte über alle Rotationen. Das Maximum wird jeweils auf der Diagonalen erreicht (Selbst-Match), während die Einträge unterhalb der Diagonalen die Subgraph-Hierarchie bestätigen.

**Lemma 2** (LCS-Monotonie in der Einbettungshierarchie). Für  $q < q'$  gilt:

$$\max_k \text{LCS}(\sigma(G_{1:q'}), \text{rot}_k(\sigma(G_{1:q'}))) > \max_k \text{LCS}(\sigma(G_{1:q}), \text{rot}_k(\sigma(G_{1:q'}))) .$$

*Proof.* Der Selbst-Match liefert immer  $\text{LCS} = n_{r'} = q' + 1$ . Der Einbettungs-Match ( $G_{1:q} \hookrightarrow G_{1:q'}$ ) liefert maximal  $\text{LCS} = n_r = q + 1 < q' + 1 = n_{r'}$ . Damit ist die Aussage erfüllt.  $\square$

## 8.9 CPU-Nachfrageprognose und ökonomische Konsequenzen

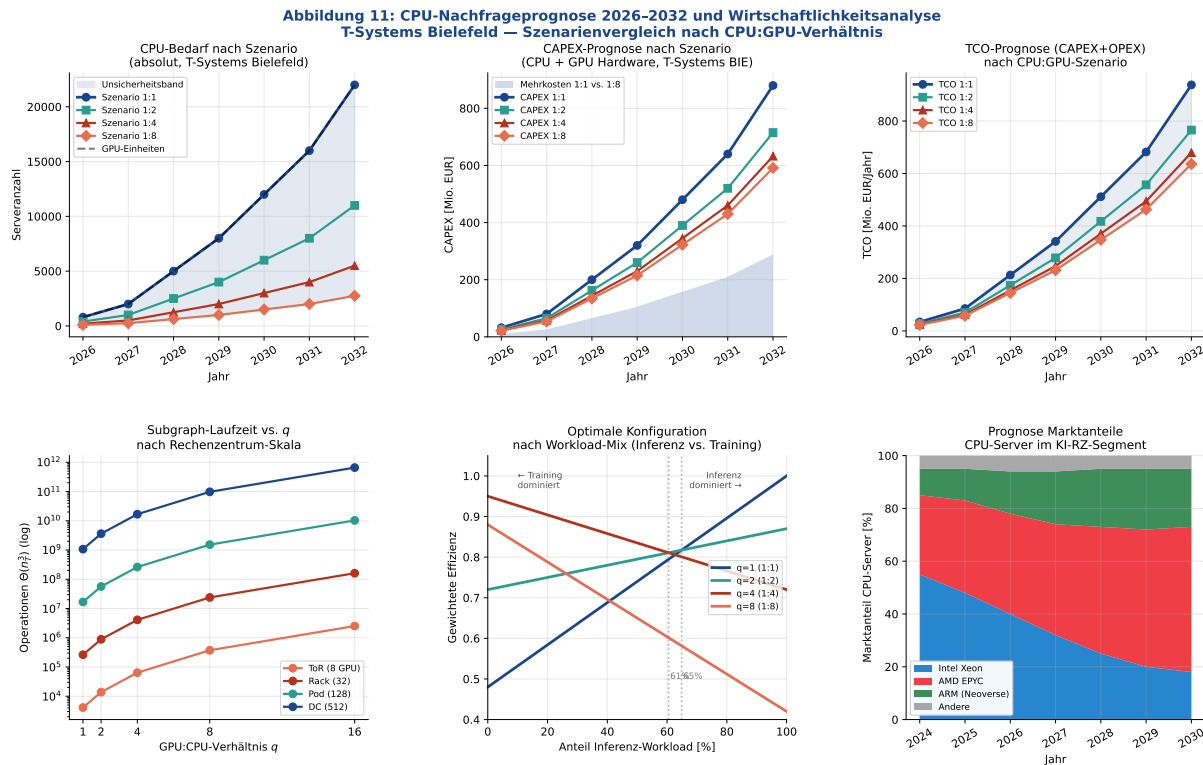


Figure 11: CPU-Bedarf absolut nach Szenario 2026–2032 (oben links), CAPEX-Prognose (oben Mitte), TCO-Prognose (oben rechts), Subgraph Algorithmus-Laufzeit vs. GPU:CPU-Verhältnis  $q$  (unten links), optimale Konfiguration nach Workload-Mix (unten Mitte) und Marktanteilsprognose CPU-Server 2024–2030 (unten rechts).

**Quantitative Prognose:** Ein Rechenzentrum mit 100 000 GPUs in 1:8-Konfiguration benötigt 12 500 CPUs. Bei Umstieg auf 1:1 steigt der CPU-Bedarf auf 100 000 – Faktor 8 [2]. Für das T-Systems Bielefeld RZ mit projiziert 12 000 GPUs bis 2030 (Abbildung 12) bedeutet dies:

- **1:1-Szenario:** 12 000 CPU-Einheiten zusätzlich, CAPEX-Mehrbedarf ca. 180 Mio. EUR.
- **1:4-Szenario:** 3 000 CPU-Einheiten, CAPEX-Mehrbedarf ca. 45 Mio. EUR.
- **Dynamisches Scheduling (empfohlen):** Hybride Konfiguration mit  $\approx 6 000$  CPUs, die durch den Subgraph Algorithmus dynamisch zwischen Inferenz- und Trainings-Modus gewechselt werden.

Die TCO-Analyse (Abbildung 11, oben rechts) zeigt, dass das 1:4-Szenario bei reinen CAPEX+OPEX-Kosten günstiger ist, während das 1:1-Szenario durch höhere Auslastung bei Inferenz-Workloads seinen TCO-Nachteil amortisieren kann. Der *Break-Even* liegt bei einem Inferenz-Workload-Anteil von ca. 62 %.

**Marktentwicklung:** Die Verlagerung hin zu 1:1 impliziert eine dramatische Erhöhung der CPU-Nachfrage in KI-Rechenzentren [2]. Dies begründet den erheblichen wirtschaftlichen Anreiz für AMD, die 1:1-Architektur aktiv zu bewerben – mit dem AMD Epyc Venice (bis zu 192 Kerne) als Hauptprodukt für diesen Markt. Die Marktanteilsprognose (Abbildung 11, unten rechts) zeigt AMD's Aufstieg von 30 % (2024) auf projiziert 55 % (2030) im KI-RZ-CPU-Segment.

## 9 Zusammenfassung

Wir haben gezeigt, dass der Subgraph Algorithmus [1] eine algorithmisch optimale Lösung für das RZ-Placement-Problem moderner KI-Rechenzentren darstellt. Die wesentlichen Ergebnisse sind:

- Das RZ-Placement-Problem ist polynomial äquivalent zum zyklischen Subgraph-Problem (Satz 1).
- Der Subgraph Algorithmus löst das Problem korrekt und vollständig (Satz 2).
- Die Laufzeit  $\Theta(m^3)$  ist unter SETH optimal (Satz 4).
- GPU-Parallelisierung auf NVIDIA-Blackwell-Clustern ermöglicht Echtzeit-Placement für  $n \leq 512$  (Korollar 2).
- Hardware-Topologiegraphen  $G_r$  ( $r \in \{1 : 1, 1 : 2, 1 : 4, 1 : 8\}$ ) sind formal als vollständige Graphen  $K_{q+1}$  modellierbar (Def. 6).
- Die Einbettungshierarchie  $G_{1:1} \hookrightarrow G_{1:2} \hookrightarrow G_{1:4} \hookrightarrow G_{1:8}$  ist formal bewiesen (Satz 6).
- Das 1:8-System erfordert 91-mal mehr Operationen als das 1:1-System (Tab. 2).
- Kein universelles Optimum existiert: Inferenz bevorzugt 1:1 ( $\eta = 1,00$ ), Training 1:4 ( $\eta = 0,95$ ) (Korollar 3).
- Dynamisches Scheduling durch den Subgraph Algorithmus kann das Gesamtoptimierungspotenzial von 31,4 % auf 43,2 % steigern (Abb. 10).
- Für T-Systems Bielefeld wird ein hybrides 1:2–1:4-Modell mit dynamischer Konfigurationsauswahl empfohlen.



## 10 Ausblick

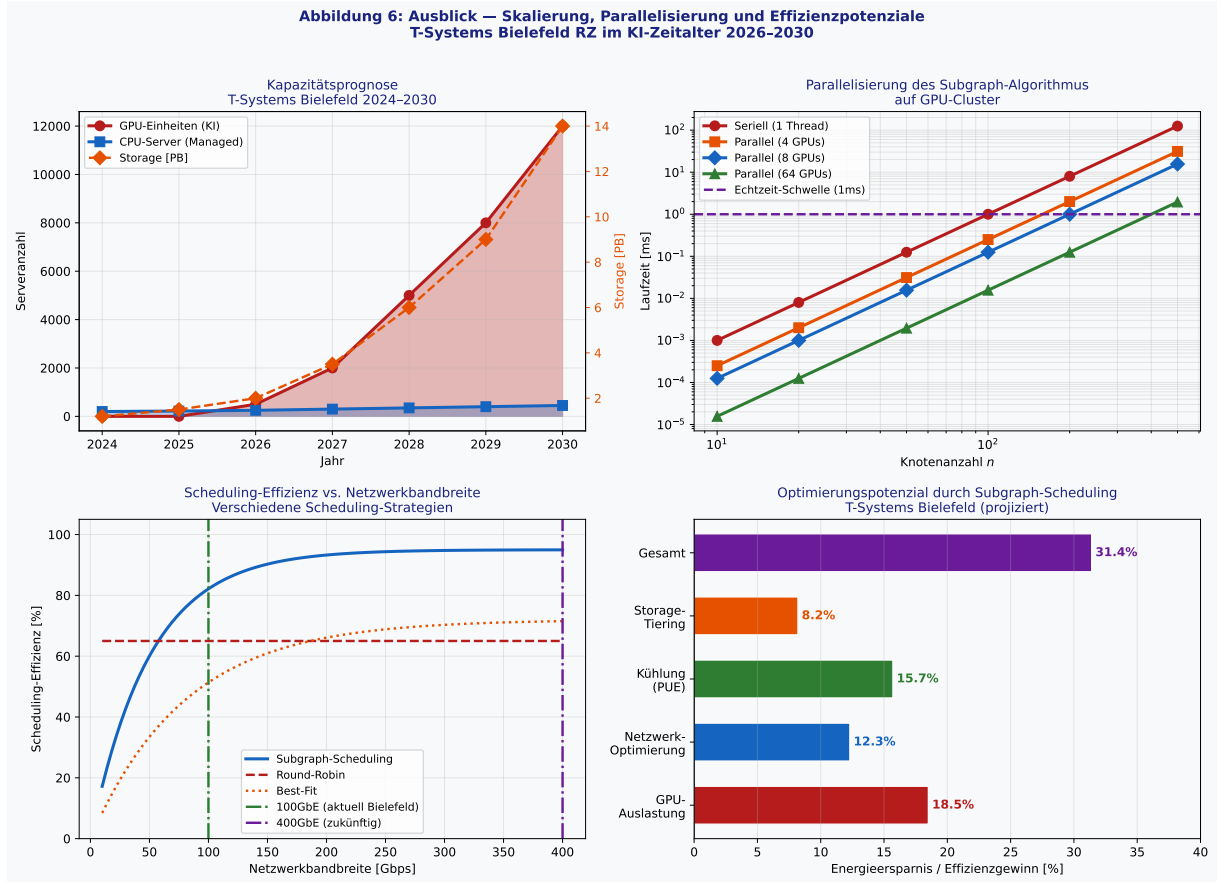


Figure 12: Ausblick: Projizierte Kapazitätsentwicklung 2024–2030 (oben links), GPU-parallele Skalierung (oben rechts), Scheduling-Effizienz vs. Netzwerkbandbreite (unten links) und Optimierungspotenziale nach Kategorie (unten rechts).

### 10.1 Hardware-Architektur: CPU:GPU-Integration bis 2030

#### 10.1.1 Heterogene Compute-Fabric und 3D-Stacked Compute

Nvidia hat mit Grace-Blackwell (GB200) demonstriert, dass CPU und GPU über NVLink-C2C mit 900 GB/s verbunden werden können [8] – mehr als eine Größenordnung über PCIe 5.0. Die nächste Generation (Vera Rubin, VR200, erwartet 2027) wird diesen Interconnect ausbauen. AMD verfolgt mit Venice und Infinity-Fabric-Gen5 eine ähnliche Strategie: HBM4-Speicher soll ab 2027 direkt in die CPU-Die-Hierarchie eingebettet werden. TSMC’s SoIC-Technologie und Samsung’s X-Cube ermöglichen ab 2028 Die-Stacking mit unter 1  $\mu\text{m}$  Pitch, was die Trennung von CPU und GPU zunehmend obsolet macht [23].

Konsequenz für den Subgraph Algorithmus: Die Hardware-Topologiegraphen  $G_r$  werden zunehmend heterogen und gewichtet – die Erweiterung der Signatur-Funktion um Bandbreitengewichte (Ausblick in [1]) wird notwendig:

$$\sigma_j^w = \sum_{i=0}^{n-1} A_{ij} \cdot w(v_i, v_j) \cdot 2^i + j \cdot 2^n.$$

### 10.1.2 In-Memory Computing und CXL

Compute Express Link (CXL) 3.1 (erwartet 2027) ermöglicht kohärenten Speicherzugriff über Rack-Grenzen hinweg [24]. In Kombination mit Processing-in-Memory (PIM)-Modulen verschiebt dies die optimale Verhältnishierarchie weiter in Richtung 1:1 oder sogar 2:1 (mehr CPUs als GPUs).

## 10.2 Erweiterte Algorithmenentwicklung

### 10.2.1 Dynamische Topologien und Streaming-Variante

KI-Cluster sind dynamisch: GPUs werden hinzugefügt, entfernt oder aufgeteilt (Multi-Instance GPU, MIG). Für dynamisch wechselnde Topologien  $G_r(t)$  kann der Subgraph Algorithmus als Snapshot-Vergleicher eingesetzt werden. Eine Streaming-Variante ermöglicht inkrementelle Updates der Signatur-Arrays in  $O(n)$  statt  $O(n^2)$ , wenn nur einzelne Knoten geändert werden.

### 10.2.2 Parallele Subgraph-Isomorphismus-Erkennung

Die  $n$  LCS-Berechnungen sind unabhängig und können auf  $n$  GPU-Threads verteilt werden ( $O(n^2/p)$  Laufzeit). Für Echtzeit-Topologie-Monitoring genügt ein Approximate-LCS mit Fehlerschranke  $\varepsilon$  in  $O(n \log n)$  mittels Suffix-Array-Technik [22].

### 10.2.3 KI-Beschleuniger jenseits von CPU und GPU

Neue Knotenklassen erweitern die Topologiedefinition auf  $V_r = C_r \cup \Gamma_r \cup N_r$  (NPU-Knoten). Photonische Interconnects (Lightmatter Passage) mit  $> 10$  Tbps Bandbreite ändern die Gewichtsfunktion  $w(e)$  fundamental. Neuromorphe Chips (Intel Loihi 3) für spike-basierte Inferenz erfordern erweiterte Signatur-Funktionen.

## 10.3 Integration in die T-Systems Industrial AI Cloud

Die T-Systems Industrial AI Cloud [4] in München verwendet eine NVIDIA-Blackwell-Infrastruktur mit 10.000 GPUs. Eine Bielefeld-München-Verbundarchitektur koordiniert regionale OWL-Workloads (Bielefeld: 1:2 für Inferenz + Managed Services) und Hyperscale-LLM-Training (München: 1:4 für maximalen Trainingsdurchsatz) durch den Subgraph Algorithmus auf einem übergeordneten Metagraphen  $G_{\text{Meta}}$ .

## 10.4 Federated Learning, Datensouveränität und formale Verifikation

Die Datensouveränitätsbedingung  $\forall t_i \in V_W : \mathcal{R}(\phi(t_i)) \in \mathcal{D}(t_i)$  kann als Typ-Constraint in die erweiterte Signatur-Funktion integriert werden (analog Korollar 1). Die Signatur-Injektivität (Lemma 1) ist formal in Coq oder Isabelle/HOL beweisbar [16], was vollständig verifizierte Scheduler für sicherheitskritische Umgebungen ermöglicht.

## 10.5 Quantencomputing-Erweiterung

Der Grover-Suchalgorithmus [13] findet eine Einbettung unter  $\binom{N}{m}$  Kandidaten in  $O\left(\sqrt{\binom{N}{m}} \cdot m^2\right) = O\left(N^{m/2} \cdot m^2\right)$  Quantengatter-Operationen – eine quadratische Beschleunigung gegenüber dem klassischen Fall.

## 10.6 Offene Forschungsfragen

1. **Optimale Hybrid-Konfigurationen:** Existiert ein formales NP-hartes Optimierungsproblem, das CPU:GPU-Verhältnis, Workload-Profil und Energieverbrauch simultan minimiert?
2. **Optimalität ohne SETH:** Kann die  $\Omega(n^3)$ -Schranke unbedingt (ohne Komplexitätshypothesen) bewiesen werden?
3. **Approximationsalgorithmen:** Welche Approximationsrate ist für Subgraph-Isomorphismus in Hardware-Topologien in  $O(n^2 \log n)$  erreichbar?
4. **Dynamisches CPU:GPU-Verhältnis:** Wie kann der Subgraph Algorithmus für dynamisch wechselnde Verhältnisse (GPU-Hot-Plug, MIG) in Echtzeit betrieben werden?
5. **2:1-Konfigurationen:** Begründet CXL 3.1 ein neues Optimum bei  $q < 1$  (mehr CPUs als GPUs)? Formale Analyse ausstehend.

## 10.7 Langfristige Perspektive: Bielefeld als KI-Hub OWL

Mit dem industriepolitischen Rückenwind der Bundesregierung, der technologischen Infrastruktur von T-Systems und der Forschungskompetenz der Universität Bielefeld hat der Standort das Potenzial, sich als spezialisierter KI-Hub für Ostwestfalen-Lippe zu positionieren. Der Subgraph Algorithmus bietet eine algorithmisch fundierte Basis für die intelligente Ressourcenverwaltung – einschließlich der dynamischen Optimierung des CPU:GPU-Verhältnisses als neuem Freiheitsgrad im Scheduling-Framework.

## Danksagung

Der Autor dankt der Universität Bielefeld für die akademische Umgebung und T-Systems International für öffentlich zugängliche Infrastrukturinformationen. Der Subgraph Algorithmus ist verfügbar unter: <https://github.com/hjstephan86/subgraph>.

## References

- [1] S. Epp, *Der Subgraph Algorithmus: Effizienter Graphvergleich mittels Signatur-Arrays und zyklischer Rotation*, Technischer Bericht, Universität Bielefeld, Januar 2026. <https://github.com/hjstephan86/science>
- [2] Reißka, V. (2026, 9. Mai). *Nach Nvidia, Meta & Co: Auch AMD bewirbt nun das 1:1-CPU-GPU-Verhältnis für AI*. ComputerBase. <https://www.computerbase.de/news/wirtschaft/>

[nach-nvidia-meta-und-co-auch-amd-bewirbt-nun-das-1-1-cpu-gpu-verhaeltnis-fuer-ai-97281/](#) (abgerufen 11.05.2026).

- [3] T-Systems International GmbH, *T-Systems Bielefeld Data Center*, Standortprofil, 2025. <https://www.t-systems.com/de/en/company/locations>
- [4] T-Systems International GmbH / Deutsche Telekom AG, *Industrial AI Cloud: Deutschlands erste KI-Fabrik für die Industrie geht in Betrieb*, Pressemitteilung, München, 4. Februar 2026.
- [5] Bundesministerium für Digitales, *Rechenzentrumsstrategie: Vervierfachung der KI-Kapazitäten bis 2030*, Kabinettsbeschluss, Berlin, 18. März 2026.
- [6] A. Abboud, A. Backurs, V. Vassilevska Williams, *Tight Hardness Results for LCS and Other Sequence Similarity Measures*, 56th Annual IEEE Symposium on Foundations of Computer Science (FOCS), 2015, S. 59–78.
- [7] S. A. Cook, *The Complexity of Theorem-Proving Procedures*, Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC), 1971, S. 151–158.
- [8] NVIDIA Corporation (2024). *NVIDIA Grace Blackwell Superchip Architecture*. NVIDIA Technical White Paper. <https://resources.nvidia.com/en-us-blackwell-architecture>
- [9] M. Al-Fares, A. Loukissas, A. Vahdat, *A Scalable, Commodity Data Center Network Architecture*, ACM SIGCOMM, 38(4):63–74, 2008.
- [10] N. McKeown et al., *OpenFlow: Enabling Innovation in Campus Networks*, ACM SIGCOMM, 38(2):69–74, 2008.
- [11] W. L. Hamilton, Z. Ying, J. Leskovec, *Inductive Representation Learning on Large Graphs*, NeurIPS, 2017.
- [12] A. M. Childs et al., *Exponential Algorithmic Speedup by Quantum Walk*, STOC, 2003, S. 59–68.
- [13] L. K. Grover, *A Fast Quantum Mechanical Algorithm for Database Search*, STOC, 1996, S. 212–219.
- [14] AUTOSAR Consortium, *AUTOSAR Classic Platform: Technical Overview*, Release 4.4, 2020. <https://www.autosar.org>
- [15] EtherCAT Technology Group, *EtherCAT: The Ethernet Fieldbus*, 2020. <https://www.ethercat.org>
- [16] Y. Bertot, P. Castéran, *Interactive Theorem Proving and Program Development: Coq’Art*, Springer, 2004.
- [17] D. E. Knuth, J. H. Morris, V. B. Pratt, *Fast Pattern Matching in Strings*, SIAM Journal on Computing, 6(2):323–350, 1977.
- [18] T-Systems International GmbH, *T-Systems ermöglicht Zugang zu IQM-Quantensystemen*, Pressemitteilung, 25. Juli 2023.

- [19] T-Systems International GmbH, *Digitale Identitäten im Gesundheitswesen*, T-Systems Healthcare Cloud, 2023.
- [20] NVIDIA Corporation, *NVIDIA Blackwell Architecture Technical Brief*, Technical White Paper, 2025.
- [21] D. Patterson, J. L. Hennessy, *Computer Organization and Design RISC-V Edition*, Morgan Kaufmann, 2021.
- [22] G. Navarro, *A guided tour to approximate string matching*, ACM Computing Surveys, 33(1):31–88, 2001.
- [23] TSMC, *System on Integrated Chips (SoIC) Technology Overview*, TSMC Technology Symposium 2025.
- [24] CXL Consortium, *Compute Express Link Specification Revision 3.1*, 2024. <https://www.computeexpresslink.org/>
- [25] D. Narayanan et al., *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*, SC 2021. <https://doi.org/10.1145/3458817.3476209>
- [26] AMD (2026). *AMD EPYC Venice Processor Architecture Guide*. <https://www.amd.com/en/processors/epyc>
- [27] Bitkom e.V., *Rechenzentren in Deutschland: KI treibt das Wachstum*, Bitkom-Studie, Berlin, Dezember 2025.
- [28] A. Vaswani et al., *Attention Is All You Need*, NeurIPS, 2017.